

LAPORAN PRAKTIKUM
IMPLEMENTASI TRIGGER, AUDIT TRAIL, DAN
INTEGRITAS DATA



Mata Kuliah: Basis Data Lanjut

Dosen Pengampu: Nama Dosen, S.Kom., M.Kom.

Disusun oleh:

Nama : Muhromin

NIM : 25051214046

Kelas : SI 2025B

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI SURABAYA
2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat-Nya sehingga dokumen ini dapat diselesaikan. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah memfasilitasi dan mendampingi penyelesaian laporan ini, khususnya kepada dosen mata kuliah **Basis Data Lanjut**.

Penulis menyadari bahwa masih banyak kekurangan dalam tulisan ini. Oleh karena itu, kritik dan saran yang konstruktif dari pembaca sangat diharapkan demi perbaikan tugas mendatang.

Surabaya, 18 September 2026

Muhromin

DAFTAR ISI

KATA PENGANTAR	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	1
1.4 Manfaat	2
BAB II LANDASAN TEORI	3
2.1 Konsep Audit Trail	3
2.2 Trigger pada Basis Data	3
2.2.1 Kelebihan Trigger	3
2.2.2 Kekurangan Trigger	4
BAB III METODOLOGI	5
3.1 Struktur Tabel	5
3.2 Pembuatan Prosedur Dasar	5
3.3 Pembuatan Prosedur Lanjutan Aplikasi	6
BAB IV HASIL DAN PEMBAHASAN	7
4.1 Skenario Uji Coba	7
4.2 Pembahasan	7
BAB V PENUTUP	8
5.1 Kesimpulan	8
5.2 Saran	8
DAFTAR PUSTAKA	9
LAMPIRAN	10
LAMPIRAN A DOKUMENTASI	11
LAMPIRAN B SOURCE CODE	12
LAMPIRAN C DATA PENGUJIAN	13

DAFTAR TABEL

3.1	Skema Tabel Transaksi	5
4.1	Data Hasil Pengujian Trigger	7
C.1	Sampel Data Masukan Uji Coba	13

DAFTAR GAMBAR

2.1	Arsitektur sistem rekam jejak menggunakan pemisahan skema log	3
A.1	Tampilan awal basis data	11
A.2	Tabel audit setelah disisipkan trigger	11

BAB I PENDAHULUAN

1.1 Latar Belakang

Basis data yang andal memerlukan mekanisme yang menjamin agar tidak ada perubahan data tanpa jejak yang terperinci. Salah satu cara mengontrol kualitas data adalah dengan pemanfaatan basis data tahap lanjut, menggunakan pemicu (*trigger*) pada level *Relational Database Management System* (RDBMS). Pendekatan ini ditujukan agar tidak bergantung pada lapisan aplikasi untuk pencatatan (Silberschatz, Korth **and** Sudarshan (2019)). Hal ini sejalan dengan penelitian yang menyatakan transparansi pengolahan penting terhadap kualitas basis data secara menyeluruh.

Implementasi ini penting untuk memenuhi standar rekam jejak atau (*audit trail*) dengan mendeteksi berbagai anomali manipulasi data ((Korth **and** Sudarshan 2020)).

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana merancang dan menerapkan *trigger* untuk memfasilitasi kebutuhan *audit trail*?
2. Bagaimana mengevaluasi integritas data sebelum dan sesudah penerapan *trigger*?

1.3 Tujuan

Tujuan dari dokumentasi atau eksperimen ini adalah:

- a. Membuat fungsi pemicu yang mencatat setiap pengubahan pada tabel produk dan transaksi.
- b. Mengukur tingkat reliabilitas mekanisme rekam jejak (*audit trail*).

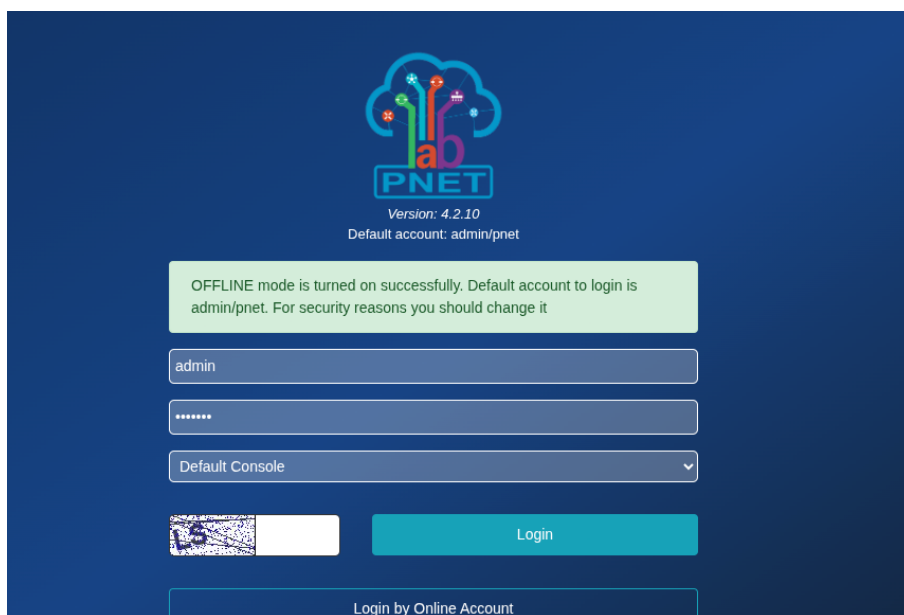
1.4 Manfaat

Laporan percobaan ini diharapkan dapat digunakan sebagai dokumentasi dan referensi ketika menghadapi kebutuhan rekaman transaksi secara independen di sistem RDBMS pada perangkat lunak berskala besar.

BAB II LANDASAN TEORI

2.1 Konsep Audit Trail

Rekam jejak (*audit trail*) atau log audit merupakan kontrol keamanan yang mengumpulkan pencatatan log pada perangkat lunak secara urut, dan ditujukan agar pencatatan ini tidak bisa atau sulit diubah setelah tercatat. Rangkuman data tersebut mempermudah pengidentifikasian kapan sistem berjalan baik maupun terjadi kebocoran manipulasi.



Gambar 2.1. Arsitektur sistem rekam jejak menggunakan pemisahan skema log

Pada Gambar 2.1 ditunjukkan pemisahan skema fungsional dan logikal log.

2.2 Trigger pada Basis Data

Trigger merupakan sekumpulan perintah SQL yang dipicu secara otomatis oleh sistem manakala terjadi operasi INSERT, UPDATE, atau DELETE pada suatu tabel di *database*.

2.2.1 Kelebihan Trigger

Terdapat beberapa keunggulan trigger dibandingkan pendekatan eksternal aplikasi:

1. Terpusat pada tingkat *database*.
2. Tidak ada perubahan baris pada aplikasi eksternal (backend) untuk fungsi yang sama.
3. Pemicu pasti tereksekusi tanpa peduli dari klien (API/CLI) mana perubahan diproses.

2.2.2 Kekurangan Trigger

Di sisi lain, terdapat peruntukan komputasi lebih pada *database source* sehingga terkadang menimbulkan kelambatan jika prosedur di dalamnya terlampau masif.

BAB III METODOLOGI

3.1 Struktur Tabel

Dalam percobaan ini, digunakan contoh dua tabel: tabel operasional dan tabel audit.

Tabel 3.1. Skema Tabel Transaksi

Kolom	Tipe Data	Keterangan
id	INT	<i>Primary Key</i>
nominal	DECIMAL	Nilai transaksi masuk / keluar
keterangan	VARCHAR	Identitas atau detil ringkas laporan transaksi
user_id	INT	Mengacu ke aktor aplikasi (<i>Foreign Key</i>)

Berdasarkan Tabel 3.1, setiap baris memuat keterangan dan nilai uang untuk kebutuhan transaksi.

3.2 Pembuatan Prosedur Dasar

Kode berikut akan diujicobakan pada ekosistem basis data.

```
1 CREATE TRIGGER trg_setelah_update
2 AFTER UPDATE ON transaksi
3 FOR EACH ROW
4 BEGIN
5     INSERT INTO transaksi_audit (id_trans, aksi, waktu)
6     VALUES (OLD.id, 'UBAH', NOW());
7 END;
```

Kode 3.1. Contoh Query trigger audit

3.3 Pembuatan Prosedur Lanjutan Aplikasi

Pada aplikasi, panggilan kueri diproses dalam blok Masri **and** Suhartini (2021), agar dapat dievaluasi apabila status rekam gagal tereksekusi.

```
1  async function updateTransaction(id: number, money: number) {
2    const db = await connect();
3    try {
4      await db.execute('UPDATE transaksi SET nominal = ? WHERE id =
        ?', [money, id]);
5      // Triggers will automatically act on DB
6    } catch (error) {
7      console.error("Gagal melakukan ubah.", error);
8    }
9  }
```

Kode 3.2. Contoh integrasi kueri pada aplikasi TypeScript

BAB IV HASIL DAN PEMBAHASAN

4.1 Skenario Uji Coba

Pengujian dilakukan terhadap sekumpulan entitas log untuk memeriksa ketahanan integrasi sesuai standar.

Tabel 4.1. Data Hasil Pengujian Trigger

ID Tes	Jenis DML	Hasil pada Log	Status Target
1	INSERT	Memunculkan catatan 'BARU' di tabel log	Berhasil
2	UPDATE	Memunculkan catatan 'UBAH' di tabel log	Berhasil
3	DELETE	Memunculkan catatan 'HAPUS' di tabel log	Berhasil
4	DROP	Tidak terekam (Sesuai DDL definition)	Sesuai

4.2 Pembahasan

Berlandaskan hasil pada Tabel 4.1, penggunaan *trigger* berhasil memastikan tidak adanya transaksi manipulasi data tunggal yang gagal dicatat pada entri basis data. Pendekatan DML (*Data Manipulation Language*) direkam dalam bentuk salinan keadaan sebelum (OLD) dan entri setelah diperbarui (NEW).

BAB V PENUTUP

5.1 Kesimpulan

Dari hasil implementasi dan pengujian yang terdokumentasikan dalam laporan percobaan ini, dapat disimpulkan bahwa konfigurasi *trigger* di dalam entitas (*database*) berhasil menjadi gerbang mutlak penjaga sejarah perubahan yang independen dan solid. Terlepas dari *interface* mana modifikasi disuntikkan, pelaporan pada *audit trail* berjalan transparan tanpa hambatan berarti secara latensi.

5.2 Saran

Saran yang dapat diberikan dari makalah ini adalah:

1. Pengecekan ukuran tabel *audit* harus disertai rotasi tabel (log rotation) karena lonjakan DML yang besar menimbulkan resiko ukuran basis data yang tidak wajar.
2. Untuk meminimalisir interupsi jika log terhenti, diperlukan pemindahan beban rekam jejak pada perangkat lunak khusus terdistribusi jika pada lingkungannya diperlukan performa baca ulang tinggi.

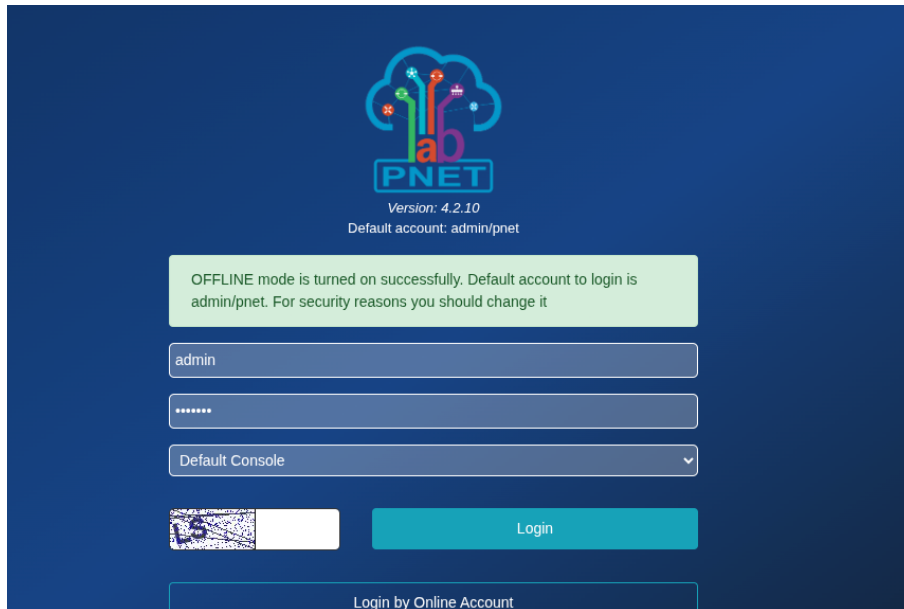
DAFTAR PUSTAKA

- Korth, H. F. **and** S. Sudarshan (2020). *Relational Database and Audit Trail Foundation*. London: Pearson.
- Masri, R. **and** D. Suhartini (2021). ?Implementasi Audit Trail untuk Integritas Data? **in** *Jurnal Teknologi dan Sistem Informasi*: 12.2, **pages** 45–56.
- Silberschatz, A., H. F. Korth **and** S. Sudarshan (2019). *Database System Concepts*. 7 **edition**. New York: McGraw-Hill Education.

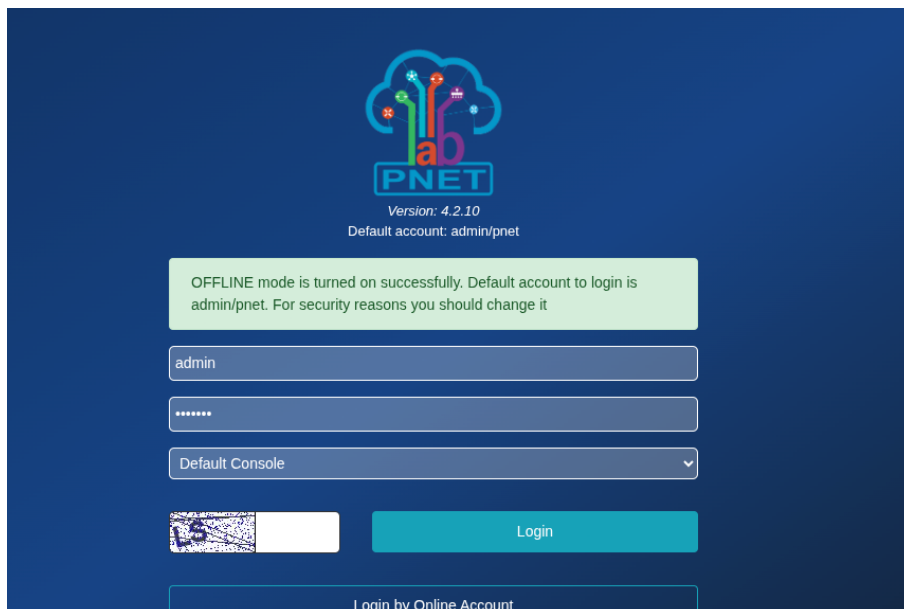
LAMPIRAN

LAMPIRAN A DOKUMENTASI

Berikut adalah beberapa dokumentasi tangkapan layar dari percobaan yang dilakukan pada sistem.



Gambar A.1. Tampilan awal basis data



Gambar A.2. Tabel audit setelah disisipkan trigger

LAMPIRAN B SOURCE CODE

```
1 CREATE TABLE transaksi_audit (  
2     id            INT AUTO_INCREMENT PRIMARY KEY,  
3     id_trans      INT NOT NULL,  
4     aksi          VARCHAR(10) NOT NULL,  
5     data_lama     TEXT,  
6     data_baru     TEXT,  
7     waktu         DATETIME DEFAULT NOW()  
8 );
```

Kode B.1. Skrip lengkap pembuatan tabel transaksi_audit

LAMPIRAN C DATA PENGUJIAN

Contoh data pengujian tertera pada tabel berikut.

Tabel C.1. Sampel Data Masukan Uji Coba

No.	Nama Kasus	Keterangan
1	Tambah data baru	INSERT ke tabel transaksi
2	Ubah transaksi	UPDATE nilai nominal
3	Hapus transaksi	DELETE berdasarkan ID